



EXAMEN PRÁCTICO: MANTENIMIENTO DE SOFTWARE

PROYECTO: CATÁLOGO DE
SMARTPHONES

CONSTRUCCIÓN Y EVOLUCIÓN
DE SOFTWARE

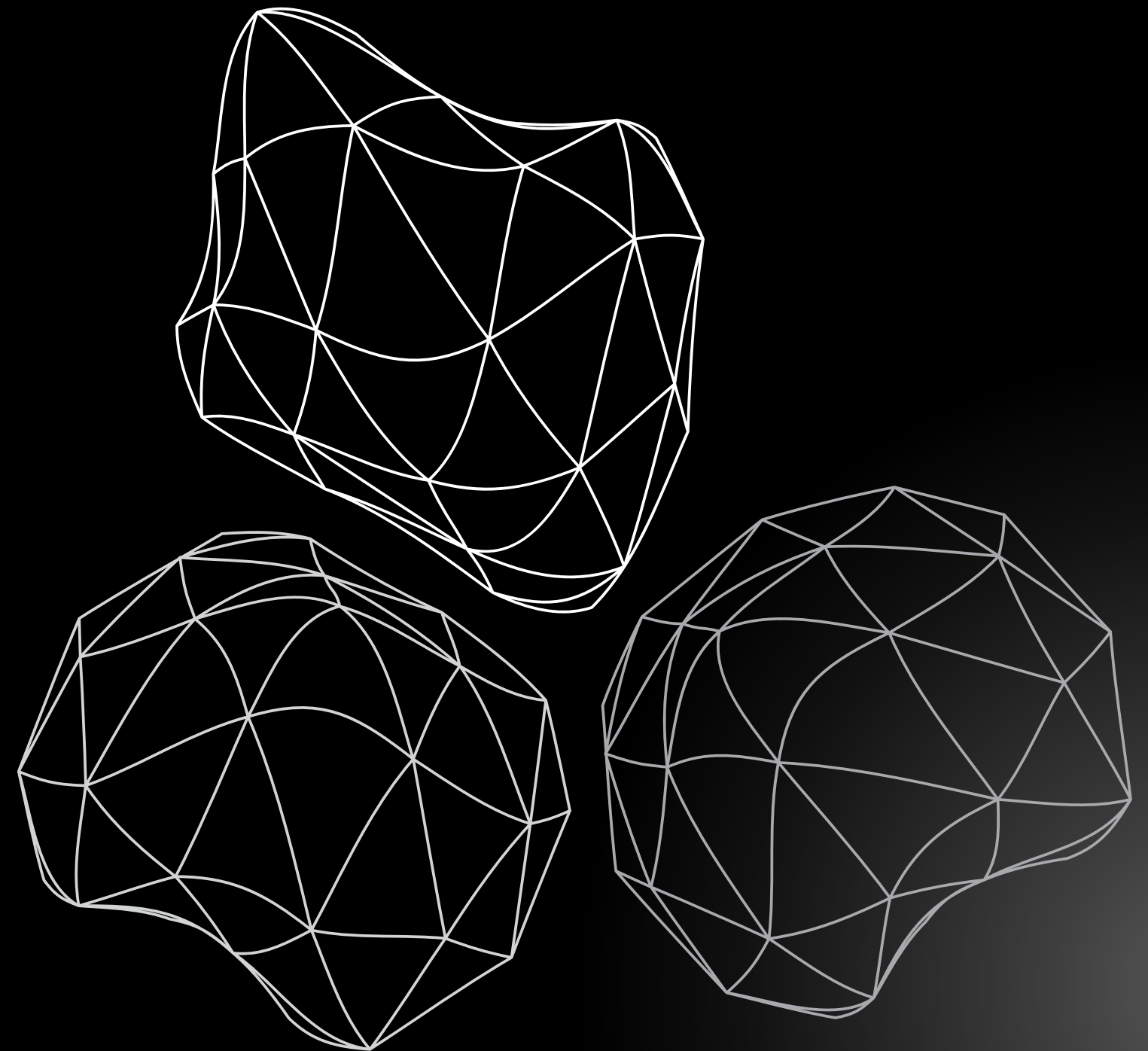
AARON CHAUCA

OBJETIVO GENERAL

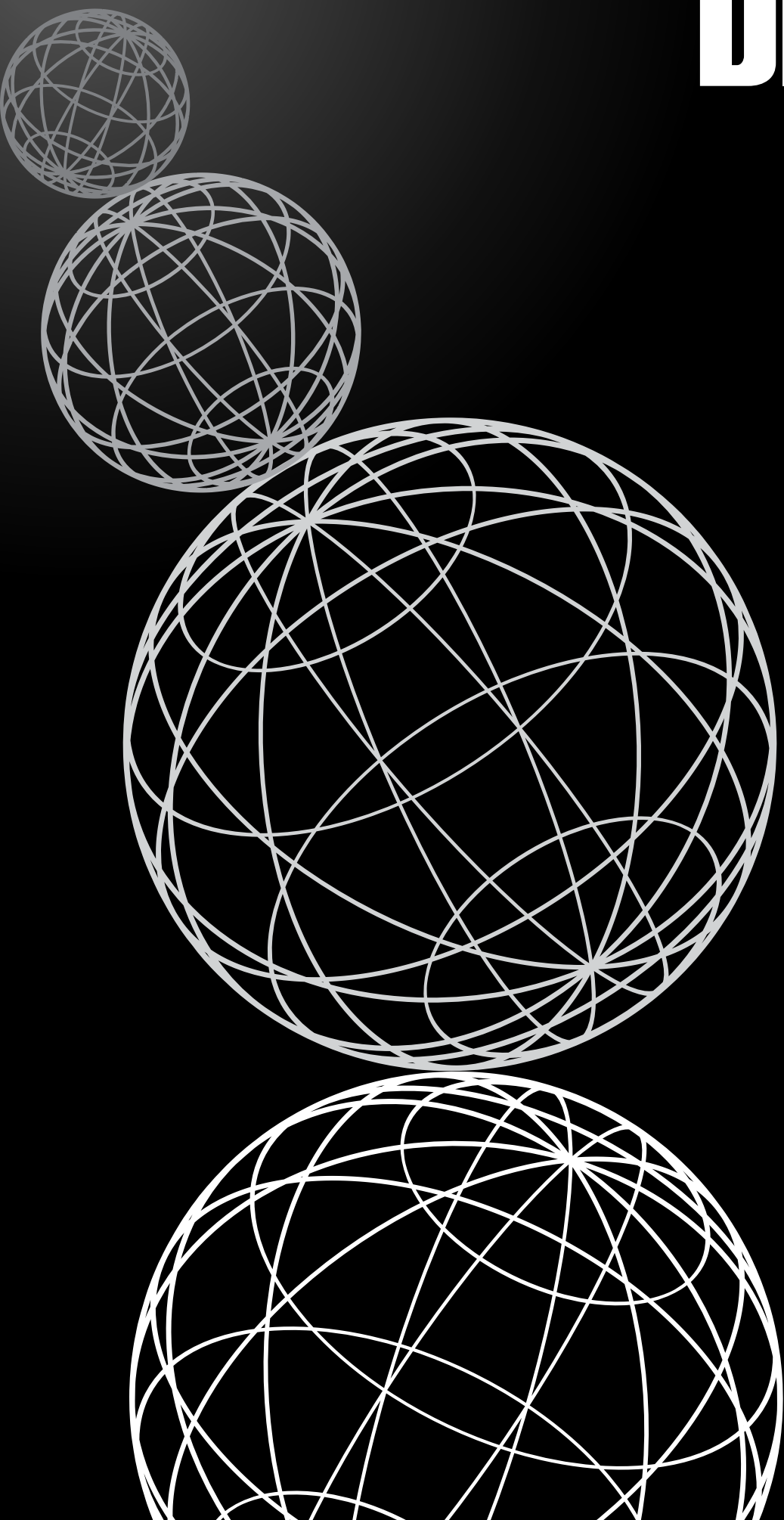
Aplicar los cuatro tipos de mantenimiento de software (correctivo, adaptativo, perfectivo y preventivo) sobre el CRUD del proyecto de catálogos de smartphones para mejorar su robustez, seguridad, mantenibilidad y calidad.

OBJETIVOS ESPECIFICOS

- Identificar deuda técnica.
- Corregir errores existentes.
- Prevenir problemas futuros.



DIAGNÓSTICO DE LA DEUDA TECNICA



Tipo	Problemas
Correctivo	Logs deficientes, falta de try-catch
Adaptativo	Sin API Key, sin ConfigModule
Perfectivo	Sin tests, sin Swagger
Preventivo	DTOs sin validaciones, ParseIntPipe ausente

MANTENIMIENTO CORRECTIVO

Problemas encontrados:

- Uso de console.log sin estructura.
- Excepciones no controladas.
- Falta de atomicidad en create smartphone.
- Eventos sin manejo de errores.

Cambios realizados:

- Logger estructurado.
- Try-catch.
- Manejo controlado de excepciones.
- Corrección del flujo create + registerEvent.

MANTENIMIENTO CORRECTIVO - LOGS ESTRUCTURADOS CON NIVELES Y TIMESTAMP

Antes

```
console.log(`CREATE smartphone ${saved.id}`);
```

```
console.log('QUERY smartphones');
```

```
console.log(`QUERY smartphone ${id}`);
```

```
console.log(`DELETE smartphone ${id}`);
```

Se reemplazó console.log por el Logger de NestJS en todos los servicios, registrando nivel INFO en operaciones exitosas, WARN cuando un recurso no existe, y ERROR ante fallos de infraestructura, siempre con timestamp ISO 8601.

Después

```
@Injectable()
export class SmartphonesService {
  // Antes: console.log sin niveles ni timestamp - ahora Logger con INFO/WARN/ERROR e ISO 8601
  private readonly logger = new Logger(SmartphonesService.name);
```

```
export class EventsService {
  // Antes: sin Logger ni try-catch - ahora Logger con INFO/ERROR y errores controlados
  private readonly logger = new Logger(EventsService.name);
```

MANTENIMIENTO CORRECTIVO

Antes

```
async create(dto: CreateSmartphoneDto): Promise<SmartphoneEntity> {
  this.validateBasicFields(dto);

  const now = new Date().toISOString();
  const smartphone = this.smartphonesRepo.create({
    brand: dto.brand.trim(),
    model: dto.model.trim(),
    price: Number(dto.price),
    storage: dto.storage.trim(),
    created_at: now,
    updated_at: now,
  });

  const saved = await this.smartphonesRepo.save(smartphone);
  console.log(`CREATE smartphone ${saved.id}`);
  await this.registerCrudEvent(
    EventAction.Create,
    'Smartphone creado',
    saved,
  );
  return saved;
}

async findAll(): Promise<SmartphoneEntity[]> {
  const smartphones = await this.smartphonesRepo.find({
    order: { id: 'ASC' },
  });
  console.log('QUERY smartphones');
  await this.registerCrudEvent(EventAction.Query, 'Smartphones consultados', {
    total: smartphones.length,
  });
  return smartphones;
}

async findOne(id: number): Promise<SmartphoneEntity> {
  const smartphone = await this.smartphonesRepo.findOneBy({ id });
  if (!smartphone) {
    throw new NotFoundException('Smartphone no encontrado');
  }
}
```

Se agregaron bloques try-catch en todos los métodos de los servicios para que un fallo de base de datos devuelva un error 500 controlado al cliente en lugar de propagar una excepción no manejada que tumbaría el servidor.

Después

```
async create(dto: CreateSmartphoneDto): Promise<SmartphoneEntity> {
  this.validateBasicFields(dto);
  // Antes: sin try-catch - ahora errores de BD quedan controlados
  try {
    const now = new Date().toISOString();
    const smartphone = this.smartphonesRepo.create({
      brand: dto.brand.trim(),
      model: dto.model.trim(),
      price: Number(dto.price),
      storage: dto.storage.trim(),
      created_at: now,
      updated_at: now,
    });

    const saved = await this.smartphonesRepo.save(smartphone);
    this.logger.log({
      operation: 'CREATE',
      entity: 'smartphone',
      id: saved.id,
      result: 'ok',
      timestamp: new Date().toISOString(),
    });
    await this.safeRegisterEvent(EventAction.Create, 'Smartphone creado', saved);
    return saved;
  } catch (error) {
    if (error instanceof BadRequestException || error instanceof NotFoundException) {
      throw error;
    }
    this.logger.error({
      operation: 'CREATE',
      entity: 'smartphone',
      result: 'error',
      message: (error as Error).message,
      timestamp: new Date().toISOString(),
    });
    throw new InternalServerErrorException('Error al crear el smartphone');
  }
}
```

MANTENIMIENTO CORRECTIVO - MANEJO DE ERRORES DE BASE DE DATOS

Antes

```
const saved = await this.smartphonesRepo.save(smartphone);
console.log(`CREATE smartphone ${saved.id}`);
await this.registerCrudEvent(
  EventAction.Create,
  'Smartphone creado',
  saved,
);
```

Se agregaron bloques try-catch en todos los métodos de los servicios para que un fallo de base de datos devuelva un error 500 controlado al cliente en lugar de propagar una excepción no manejada que tumbaría el servidor.

Después

```
async create(dto: CreateSmartphoneDto): Promise<SmartphoneEntity> {
  this.validateBasicFields(dto);
  // Antes: sin try-catch - ahora errores de BD quedan controlados
  try {
    const now = new Date().toISOString();
    const smartphone = this.smartphonesRepo.create({
      brand: dto.brand.trim(),
      model: dto.model.trim(),
      price: Number(dto.price),
      storage: dto.storage.trim(),
      created_at: now,
      updated_at: now,
    });

    const saved = await this.smartphonesRepo.save(smartphone);
    this.logger.log({
      operation: 'CREATE',
      entity: 'smartphone',
      id: saved.id,
      result: 'ok',
      timestamp: new Date().toISOString(),
    });
    await this.safeRegisterEvent(EventAction.Create, 'Smartphone creado', saved);
    return saved;
  } catch (error) {
    if (error instanceof BadRequestException || error instanceof NotFoundException) {
      throw error;
    }
    this.logger.error({
      operation: 'CREATE',
      entity: 'smartphone',
      result: 'error',
      message: (error as Error).message,
      timestamp: new Date().toISOString(),
    });
    throw new InternalServerErrorException('Error al crear el smartphone');
  }
}
```


MANTENIMIENTO CORRECTIVO - BUG DE PROPAGACIÓN EN REGISTRO DE EVENTOS

Antes

```
async registerEvent(dto: CreateEventDto): Promise<RegisterEventResult> {
  const payloadStr = JSON.stringify(dto.payload ?? {});
  const event = this.eventsRepo.create({
    source: dto.source,
    entity: dto.entity,
    action: dto.action,
    title: dto.title,
    description: dto.description ?? null,
    payload: payloadStr,
    created_at: new Date().toISOString(),
  });

  await this.eventsRepo.save(event);
  return { ok: true };
}

async findAll(): Promise<EventEntity[]> {
  return this.eventsRepo.find({
    order: { created_at: 'DESC', id: 'DESC' },
  });
}

async findBySource(source: string): Promise<EventEntity[]> {
  return this.eventsRepo.find({
    where: { source },
    order: { created_at: 'DESC', id: 'DESC' },
  });
}
```

Se corrigió el bug donde un fallo al guardar el evento de auditoría cancelaba la operación CRUD aunque el dato ya había sido persistido. Ahora el fallo del evento se registra como WARN sin afectar la respuesta al cliente.

Después

```
async registerEvent(dto: CreateEventDto): Promise<RegisterEventResult> {
  // Antes: sin try-catch - ahora error de persistencia controlado y logueado
  try {
    const payloadStr = JSON.stringify(dto.payload ?? {});
    const event = this.eventsRepo.create({
      source: dto.source,
      entity: dto.entity,
      action: dto.action,
      title: dto.title,
      description: dto.description ?? null,
      payload: payloadStr,
      created_at: new Date().toISOString(),
    });

    await this.eventsRepo.save(event);
    this.logger.log({
      operation: 'REGISTER_EVENT',
      action: dto.action,
      entity: dto.entity,
      result: 'ok',
      timestamp: new Date().toISOString(),
    });
    return { ok: true };
  } catch (error) {
    this.logger.error({
      operation: 'REGISTER_EVENT',
      action: dto.action,
      entity: dto.entity,
      result: 'error',
      message: (error as Error).message,
      timestamp: new Date().toISOString(),
    });
    throw new InternalServerErrorException('Error al registrar el evento');
  }
}
```


MANTENIMIENTO ADAPTATIVO

Problemas:

- Endpoints públicos.
- No existía API Key.
- Variables de entorno incompletas.

Solución:

- API Key Guard.
- ConfigModule.
- Archivo .env.
- Validación de variables al arranque.

MANTENIMIENTO ADAPTATIVO - GUARD DE API KEY EN SMARTPHONES

Antes

```
@Controller('api/smartphones')
export class SmartphonesController {
  constructor(private readonly smartphonesService: SmartphonesService) {}

  @Post()
  create(@Body() dto: CreateSmartphoneDto) {
    return this.smartphonesService.create(dto);
  }

  @Get()
  findAll() {
    return this.smartphonesService.findAll();
  }

  @Get(':id')
  findOne(@Param('id') id: string) {
    return this.smartphonesService.findOne(Number(id));
  }

  @Put(':id')
  update(@Param('id') id: string, @Body() dto: UpdateSmartphoneDto) {
    return this.smartphonesService.update(Number(id), dto);
  }

  @Delete(':id')
  remove(@Param('id') id: string) {
    return this.smartphonesService.remove(Number(id));
  }
}
```

Se aplicó
ApiKeyGuard a los 5
endpoints del CRUD
de smartphones.
Cualquier solicitud
sin el header X-FIS-
EPN-KEY correcto
recibe un error 401.

Después

```
// Antes: endpoints públicos sin ninguna protección
// Ahora: todos los endpoints requieren header X-FIS-EPN-KEY via ApiKeyGuard
@ApiTags('Smartphones')
@ApiSecurity('x-fis-epn-key')
@UseGuards(ApiKeyGuard)
@Controller('api/smartphones')
export class SmartphonesController {
  constructor(private readonly smartphonesService: SmartphonesService) {}

  @Post()
  create(@Body() dto: CreateSmartphoneDto) {
    return this.smartphonesService.create(dto);
  }

  @Get()
  findAll() {
    return this.smartphonesService.findAll();
  }

  // Antes: Number(id) sin validación, NaN llegaba al service
  // Ahora: ParseIntPipe rechaza con 400 cualquier valor que no sea entero
  @Get(':id')
  findOne(@Param('id', ParseIntPipe) id: number) {
    return this.smartphonesService.findOne(id);
  }

  @Put(':id')
  update(@Param('id', ParseIntPipe) id: number, @Body() dto: UpdateSmartphoneDto) {
    return this.smartphonesService.update(id, dto);
  }

  @Delete(':id')
  remove(@Param('id', ParseIntPipe) id: number) {
    return this.smartphonesService.remove(id);
  }
}
```

MANTENIMIENTO ADAPTATIVO - GUARD DE API KEY EN EVENTS

Antes

```
@Controller('events')
export class EventsController {
  constructor(private readonly eventsService: EventsService) {}

  @Post()
  registerEvent(@Body() dto: CreateEventDto) {
    return this.eventsService.registerEvent(dto);
  }

  @Get()
  findAll() {
    return this.eventsService.findAll();
  }

  @Get('source/:source')
  findBySource(@Param('source') source: string) {
    return this.eventsService.findBySource(source);
  }

  @Get('entity/:entity')
  findByEntity(@Param('entity') entity: string) {
    return this.eventsService.findByEntity(entity);
  }
}
```

Se protegieron los 4 endpoints del módulo de eventos con ApiKeyGuard.

Después

```
// Antes: endpoints de eventos públicos sin protección
// Ahora: todos requieren header X-FIS-EPN-KEY via ApiKeyGuard
@ApiTags('Events')
@ApiSecurity('x-fis-epn-key')
@UseGuards(ApiKeyGuard)
@Controller('events')
export class EventsController {
  constructor(private readonly eventsService: EventsService) {}

  @Post()
  registerEvent(@Body() dto: CreateEventDto) {
    return this.eventsService.registerEvent(dto);
  }

  @Get()
  findAll() {
    return this.eventsService.findAll();
  }

  @Get('source/:source')
  findBySource(@Param('source') source: string) {
    return this.eventsService.findBySource(source);
  }

  @Get('entity/:entity')
  findByEntity(@Param('entity') entity: string) {
    return this.eventsService.findByEntity(entity);
  }
}
```


MANTENIMIENTO ADAPTATIVO - GUARD DE API

KEY EN HEALTH Y STATS

Antes

```
@Controller('health')
export class HealthController {
  constructor(private readonly dataSource: DataSource) {}

  @Get()
  async check() {
    const timestamp = new Date().toISOString();

    try {
      await this.dataSource.query('SELECT 1');
      return { status: 'ok', database: 'ok', timestamp };
    } catch {
      throw new ServiceUnavailableException({
        status: 'error',
        database: 'down',
        timestamp,
      });
    }
  }
}
```

Se protegieron los endpoints de salud del sistema y estadísticas con ApiKeyGuard.

Después

```
// Antes: sin protección ni logs - ahora protegido con ApiKeyGuard y Logger estructurado
@ApiTags('Health')
@ApiSecurity('x-fis-epn-key')
@UseGuards(ApiKeyGuard)
@Controller('health')
export class HealthController {
  private readonly logger = new Logger(HealthController.name);

  constructor(private readonly dataSource: DataSource) {}

  @Get()
  async check() {
    const timestamp = new Date().toISOString();

    try {
      await this.dataSource.query('SELECT 1');
      this.logger.log({ operation: 'HEALTH_CHECK', result: 'ok', timestamp });
      return { status: 'ok', database: 'ok', timestamp };
    } catch {
      this.logger.error({
        operation: 'HEALTH_CHECK',
        result: 'error',
        database: 'down',
        timestamp,
      });
      throw new ServiceUnavailableException({
        status: 'error',
        database: 'down',
        timestamp,
      });
    }
  }
}
```

MANTENIMIENTO ADAPTATIVO - VARIABLES DE ENTORNO Y ARCHIVO .ENV

Antes

```
@Module({
  imports: [
    TypeOrmModule.forRoot({
      type: 'better-sqlite3',
      database: process.env.DB_PATH ?? 'db/events.sqlite',
      entities: [EventEntity, SmartphoneEntity],
      migrations: [
        CreateEventsTable1760000000000,
        CreateSmartphonesTable1760000001000,
      ],
      migrationsRun: process.env.DB_MIGRATIONS_RUN !== 'false',
      synchronize: process.env.DB_SYNCHRONIZE === 'true',
    }),
  ],
  exports: [TypeOrmModule],
})
export class DatabaseModule {}
```

Después

```
1  # Antes: variables de entorno no documentadas ni centralizadas
2  # Ahora: archivo .env con todas las variables requeridas por el sistema
3
4  PORT=3000
5  DB_PATH=db/events.sqlite
6  DB_MIGRATIONS_RUN=true
7  DB_SYNCHRONIZE=false
8
9  # Clave de acceso requerida en header X-FIS-EPN-KEY para todos los endpoints
10 API_KEY=epn-fis-2026-secret-key
11
```

Se creó el archivo .env con todas las variables del sistema (PORT, DB_PATH, API_KEY, entre otras), se añadió .env.example como plantilla pública y se excluyó .env del repositorio vía .gitignore.

MANTENIMIENTO ADAPTATIVO - ARRANQUE SEGURO Y DOCUMENTACIÓN API

Antes

```
async function bootstrap() {
  const app = await NestFactory.create<NestExpressApplication>(AppModule);
  app.useStaticAssets(join(__dirname, '..', 'public'));
  app.useGlobalPipes(
    new ValidationPipe({
      forbidNonWhitelisted: true,
      transform: true,
      whitelist: true,
    }),
  );
  await app.listen(process.env.PORT ?? 3000);
}
void bootstrap();
```

Se mejoró el arranque de la aplicación agregando manejo limpio de apagado con `enableShutdownHooks()` para señales como `SIGTERM/SIGINT`, y se incorporó Swagger en `/api` para documentar y probar los endpoints del CRUD de smartphones con autenticación por API Key.

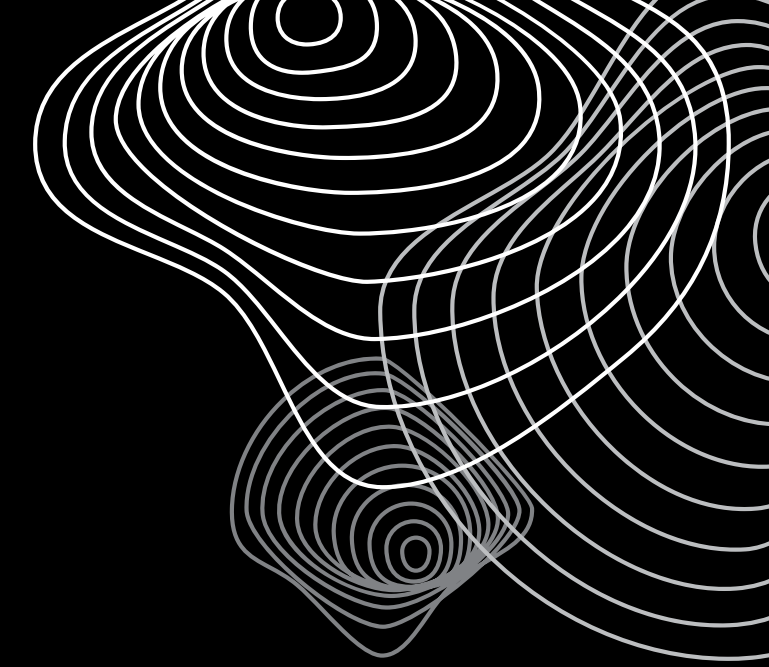
Después

```
async function bootstrap() {
  const app = await NestFactory.create<NestExpressApplication>(AppModule);
  // Antes: sin shutdown hooks – ahora maneja SIGTERM/SIGINT limpiamente al desplegar
  app.enableShutdownHooks();
  app.useStaticAssets(join(__dirname, '..', 'public'));
  app.useGlobalPipes(
    new ValidationPipe({
      forbidNonWhitelisted: true,
      transform: true,
      whitelist: true,
    }),
  );

  // Antes: sin documentación de API – ahora Swagger disponible en /api
  const config = new DocumentBuilder()
    .setTitle('EPN Event Manager API')
    .setDescription('API CRUD de smartphones con registro de eventos')
    .setVersion('1.0')
    .addApiKey(
      { type: 'apiKey', name: 'x-fis-epn-key', in: 'header' },
      'x-fis-epn-key',
    )
    .build();
  const document = SwaggerModule.createDocument(app, config);
  SwaggerModule.setup('api', app, document);

  await app.listen(process.env.PORT ?? 3000);
}
void bootstrap();
```


MANTENIMIENTO PERFECTIVO

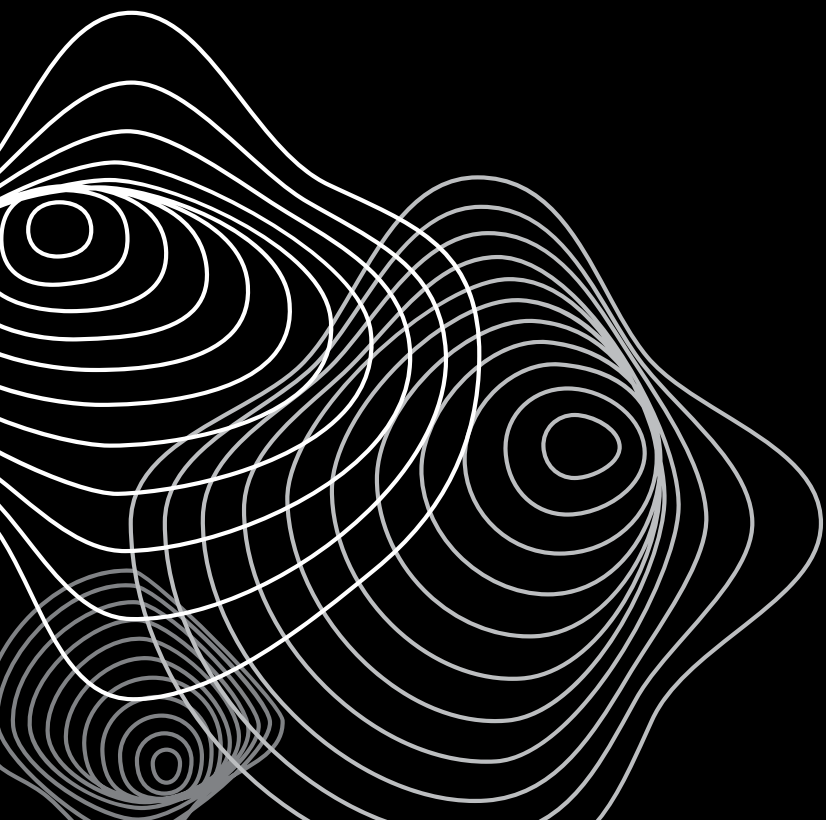


Problemas:

- Sin pruebas unitarias.
- Sin documentación API.
- Código plantilla innecesario.

Mejoras:

- Tests unitarios.
- Swagger.
- Eliminación de código muerto.



MANTENIMIENTO PERFECTIVO - PRUEBAS UNITARIAS DE LÓGICA DE NEGOCIO

Antes

```
describe('AppController', () => {
  let appController: AppController;

  beforeEach(async () => {
    const app: TestingModule = await Test.createTestingModule({
      controllers: [AppController],
      providers: [AppService],
    }).compile();

    appController = app.get<AppController>(AppController);
  });

  describe('root', () => {
    it('should return "Hello World!"', () => {
      expect(appController.getHello()).toBe('Hello World!');
    });
  });
});
```

Después

```
const mockSmartphone: SmartphoneEntity = {
  id: 1,
  brand: 'Apple',
  model: 'iPhone 15',
  price: 999.99,
  storage: '128GB',
  created_at: '2026-05-29T00:00:00.000Z',
  updated_at: '2026-05-29T00:00:00.000Z',
};

const mockRepo = {
  create: jest.fn(),
  save: jest.fn(),
  find: jest.fn(),
  findOneBy: jest.fn(),
  remove: jest.fn()
}
```

Se crearon pruebas unitarias sobre SmartphonesService cubriendo los cinco métodos CRUD y los seis casos de validación de validateBasicFields (brand, model, price cero, price negativo, storage vacío).

MANTENIMIENTO PERFECTIVO - DOCUMENTACIÓN DE API CON SWAGGER

Antes

No existe
@nestjs/swagger
en el "package.json"

Se integró @nestjs/swagger y se configuró la interfaz disponible en /api con autenticación por API Key. Todos los DTOs fueron anotados con @ApiProperty para mostrar ejemplos de cada campo.

Después

```
"dependencies": {  
  "@nestjs/common": "^11.0.1",  
  "@nestjs/core": "^11.0.1",  
  "@nestjs/platform-express": "^11.0.1",  
  "@nestjs/swagger": "^11.4.4",  
  "@nestjs/typeorm": "^11.0.1",  
  "better-sqlite3": "^12.9.0",  
  "class-transformer": "^0.5.1",  
  "class-validator": "^0.15.1",  
  "reflect-metadata": "^0.2.2",  
  "rxjs": "^7.8.1",  
  "swagger-ui-express": "^5.0.1",  
  "typeorm": "^0.3.28"  
},
```


MANTENIMIENTO PERFECTIVO

Antes

```
@Controller()
export class AppController {
  constructor(private readonly appService: AppService) {}

  @Get()
  getHello(): string {
    return this.appService.getHello();
  }
}
```

Se aplicó ApiKeyGuard a los endpoints del CRUD de smartphones. Cualquier solicitud sin el header X-FIS-EPN-KEY correcto recibe un error 401.

Después

```
// Antes: endpoints públicos sin ninguna protección
// Ahora: todos los endpoints requieren header X-FIS-EPN-KEY via ApiKeyGuard
@ApiTags('Smartphones')
@ApiSecurity('x-fis-epn-key')
@UseGuards(ApiKeyGuard)
@Controller('api/smartphones')
export class SmartphonesController {
  constructor(private readonly smartphonesService: SmartphonesService) {}

  @Post()
  create(@Body() dto: CreateSmartphoneDto) {
    return this.smartphonesService.create(dto);
  }

  @Get()
  findAll() {
    return this.smartphonesService.findAll();
  }
}
```

MANTENIMIENTO PREVENTIVO

Problemas:

- Sin límites de longitud.
- Sin validaciones numéricas.
- IDs inválidos aceptados.

Solución:

- MaxLength.
- Min.
- ParseIntPipe.
- Validaciones adicionales.

MANTENIMIENTO PREVENTIVO - MAXLENGTH EN DTO DE CREACIÓN

Antes

```
export class CreateSmartphoneDto {  
  @IsString()  
  @NotEmpty()  
  brand: string;  
  
  @IsString()  
  @NotEmpty()  
  model: string;  
  
  @IsNumber()  
  price: number;  
  
  @IsString()  
  @NotEmpty()  
  storage: string;  
}
```

Se agregaron
@MaxLength(100) en
brand,
@MaxLength(150) en
model y
@MaxLength(50) en
storage para evitar que
cadenas excesivamente
largas lleguen a la base
de datos.

Después

```
export class CreateSmartphoneDto {  
  // Antes: sin MaxLength - ahora límite 100 chars para prevenir entradas maliciosas  
  @ApiProperty({ example: 'Samsung', maxLength: 100 })  
  @IsString()  
  @NotEmpty()  
  @MaxLength(100)  
  brand!: string;  
  
  // Antes: sin MaxLength - ahora límite 150 chars  
  @ApiProperty({ example: 'Galaxy S24', maxLength: 150 })  
  @IsString()  
  @NotEmpty()  
  @MaxLength(150)  
  model!: string;  
  
  // Antes: @Min solo validado en service - ahora también validado en el DTO  
  @ApiProperty({ example: 799.99, minimum: 0.01 })  
  @IsNumber()  
  @Min(0.01)  
  price!: number;  
  
  // Antes: sin MaxLength - ahora límite 50 chars  
  @ApiProperty({ example: '128GB', maxLength: 50 })  
  @IsString()  
  @NotEmpty()  
  @MaxLength(50)  
  storage!: string;  
}
```

MANTENIMIENTO PREVENTIVO - MAXLENGTH EN DTO DE ACTUALIZACIÓN

Antes

```
export class UpdateSmartphoneDto {  
  @IsString()  
  @IsOptional()  
  brand?: string;  
  
  @IsString()  
  @IsOptional()  
  model?: string;  
  
  @IsNumber()  
  @IsOptional()  
  price?: number;  
  
  @IsString()  
  @IsOptional()  
  storage?: string;  
}
```

Se aplicaron los mismos límites de longitud que en el DTO de creación para mantener consistencia en la validación de entradas.

Después

```
export class UpdateSmartphoneDto {  
  // Antes: sin MaxLength – ahora límites consistentes con CreateSmartphoneDto  
  @ApiPropertyOptional({ example: 'Apple', maxLength: 100 })  
  @IsString()  
  @IsOptional()  
  @MaxLength(100)  
  brand?: string;  
  
  @ApiPropertyOptional({ example: 'iPhone 15', maxLength: 150 })  
  @IsString()  
  @IsOptional()  
  @MaxLength(150)  
  model?: string;  
  
  @ApiPropertyOptional({ example: 999.99, minimum: 0.01 })  
  @IsNumber()  
  @IsOptional()  
  @Min(0.01)  
  price?: number;  
  
  @ApiPropertyOptional({ example: '256GB', maxLength: 50 })  
  @IsString()  
  @IsOptional()  
  @MaxLength(50)  
  storage?: string;  
}
```


MANTENIMIENTO PREVENTIVO - PARSEINTPIPE

EN PARÁMETROS DE ID

Antes

```
@Get('/:id')
findOne(@Param('id') id: string) {
  return this.smartphonesService.findOne(Number(id));
}

@Put('/:id')
update(@Param('id') id: string, @Body() dto: UpdateSmartphoneDto) {
  return this.smartphonesService.update(Number(id), dto);
}

@Delete('/:id')
remove(@Param('id') id: string) {
  return this.smartphonesService.remove(Number(id));
}
}
```

Se reemplazó Number(id) por ParseIntPipe en los tres endpoints que reciben un ID. Ahora valores como "abc" o "-1" son rechazados con un 400 antes de llegar al servicio.

Después

```
// Antes: Number(id) sin validación, NaN llegaba al service
// Ahora: ParseIntPipe rechaza con 400 cualquier valor que no sea entero
@Get('/:id')
findOne(@Param('id', ParseIntPipe) id: number) {
  return this.smartphonesService.findOne(id);
}

@Put('/:id')
update(@Param('id', ParseIntPipe) id: number, @Body() dto: UpdateSmartphoneDto) {
  return this.smartphonesService.update(id, dto);
}

@Delete('/:id')
remove(@Param('id', ParseIntPipe) id: number) {
  return this.smartphonesService.remove(id);
}
}
```

MANTENIMIENTO PREVENTIVO – MAXLENGTH EN PARÁMETROS DE EVENTOS

Antes

```
export class CreateEventDto {
  @IsString()
  @IsNotEmpty()
  source: string;

  @IsString()
  @IsNotEmpty()
  entity: string;

  @IsEnum(EventAction)
  action: EventAction;

  @IsString()
  @IsNotEmpty()
  title: string;

  @IsString()
  @IsOptional()
  description?: string;

  @IsObject()
  @IsOptional()
  payload?: Record<string, unknown>;
}
```

Se agregó
@MaxLength(100) a los
campos source y entity del
DTO de eventos para
limitar el tamaño de los
parámetros que llegan a
las consultas de base de
datos.

Después

```
export class CreateEventDto {
  // Antes: sin MaxLength – ahora límites para prevenir payloads excesivamente largos
  @ApiProperty({ example: 'epn-event-manager', maxLength: 100 })
  @IsString()
  @IsNotEmpty()
  @MaxLength(100)
  source!: string;

  @ApiProperty({ example: 'smartphone', maxLength: 100 })
  @IsString()
  @IsNotEmpty()
  @MaxLength(100)
  entity!: string;

  @ApiProperty({ enum: EventAction })
  @IsEnum(EventAction)
  action!: EventAction;

  @ApiProperty({ example: 'Smartphone creado', maxLength: 200 })
  @IsString()
  @IsNotEmpty()
  @MaxLength(200)
  title!: string;

  @ApiPropertyOptional({ example: 'Descripción del evento', maxLength: 500 })
  @IsString()
  @IsOptional()
  @MaxLength(500)
  description?: string;

  @ApiPropertyOptional({ example: { id: 1 } })
  @IsObject()
  @IsOptional()
  payload?: Record<string, unknown>;
}
```

MANTENIMIENTO PREVENTIVO - CIERRE LIMPIO DEL SERVIDOR

Antes

```
async function bootstrap() {
  const app = await NestFactory.create<NestExpressApplication>(AppModule);
  app.useStaticAssets(join(__dirname, '..', 'public'));
  app.useGlobalPipes(
    new ValidationPipe({
      forbidNonWhitelisted: true,
      transform: true,
      whitelist: true,
    }),
  );
  await app.listen(process.env.PORT ?? 3000);
}
void bootstrap();
```

Se activó `app.enableShutdownHooks()` en `main.ts` para que la aplicación responda correctamente a señales como `SIGTERM` o `SIGINT`, permitiendo apagar el servidor de forma ordenada y reduciendo el riesgo de interrupciones bruscas.

Después

```
async function bootstrap() {
  const app = await NestFactory.create<NestExpressApplication>(AppModule);
  // Antes: sin shutdown hooks – ahora maneja SIGTERM/SIGINT limpiamente al desplegar
  app.enableShutdownHooks();
  app.useStaticAssets(join(__dirname, '..', 'public'));
  app.useGlobalPipes(
    new ValidationPipe({
      forbidNonWhitelisted: true,
      transform: true,
      whitelist: true,
    }),
  );

  // Antes: sin documentación de API – ahora Swagger disponible en /api
  const config = new DocumentBuilder()
    .setTitle('EPN Event Manager API')
    .setDescription('API CRUD de smartphones con registro de eventos')
    .setVersion('1.0')
    .addApiKey(
      { type: 'apiKey', name: 'x-fis-eqn-key', in: 'header' },
      'x-fis-eqn-key',
    )
    .build();
  const document = SwaggerModule.createDocument(app, config);
  SwaggerModule.setup('api', app, document);

  await app.listen(process.env.PORT ?? 3000);
}
void bootstrap();
```

PRUEBA DE VIDA

```
PS C:\Users\aaaron\Escritorio\ChauquisPro\ConstrucionYEvo\carpetaParaExamen\CHAUCA-ROJAS-AARON-ERNESTO-constswgr2\Presentation-codigo-EXAMEN\epn-event-manager\epn-event-manager> npm run start:dev
```

Smartphones

CRUD persistente con auditoria de eventos

Actualizar

Crear smartphone

Marca

Modelo

Precio

Almacenamiento

Guardar

Catalogo

ID	Marca	Modelo	Precio	Storage	
1	Xiaomi	redmi	\$300.00	256 gb	<button>Editar</button> <button>Eliminar</button>

Smartphones

CRUD persistente con auditoria de eventos

Actualizar

Smartphone creado.

Crear smartphone

Marca

Modelo

Precio

Almacenamiento

Guardar

Catalogo

ID	Marca	Modelo	Precio	Storage	
1	Xiaomi	redmi	\$300.00	256 gb	<button>Editar</button> <button>Eliminar</button>
2	Iphone	17 Pro Max	\$1500.00	256 gb	<button>Editar</button> <button>Eliminar</button>

PRUEBA DE VIDA

Smartphones

CRUD persistente con auditoria de eventos

Actualizar

Smartphone actualizado.

Crear smartphone

Marca

Modelo

Precio

Almacenamiento

Guardar

Catalogo

ID	Marca	Modelo	Precio	Storage		
1	Xiaomi	redmi	\$500.00	256 gb	Editar	Eliminar
2	Iphone	17 Pro Max	\$1500.00	256 gb	Editar	Eliminar

Smartphones

CRUD persistente con auditoria de eventos

Actualizar

Smartphone eliminado.

Crear smartphone

Marca

Modelo

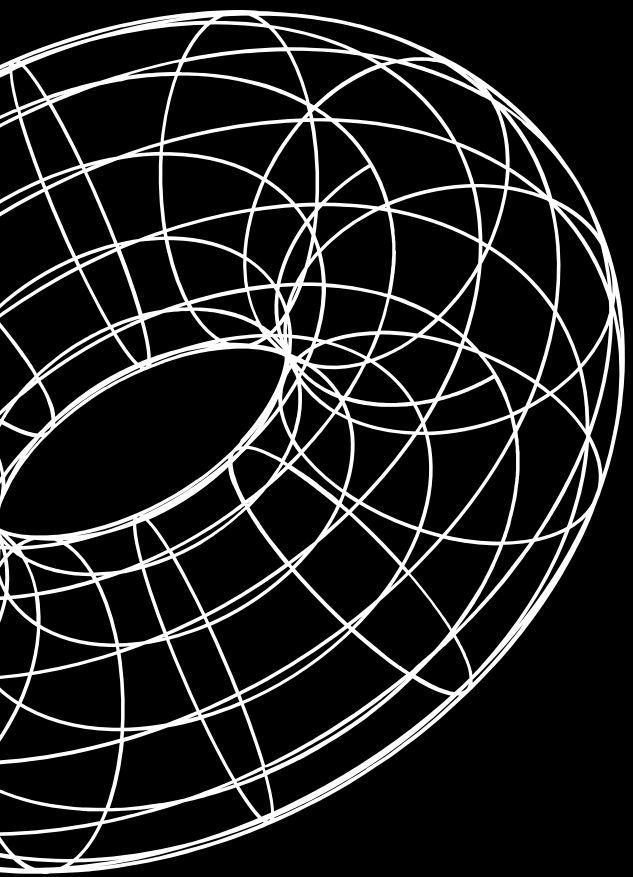
Precio

Almacenamiento

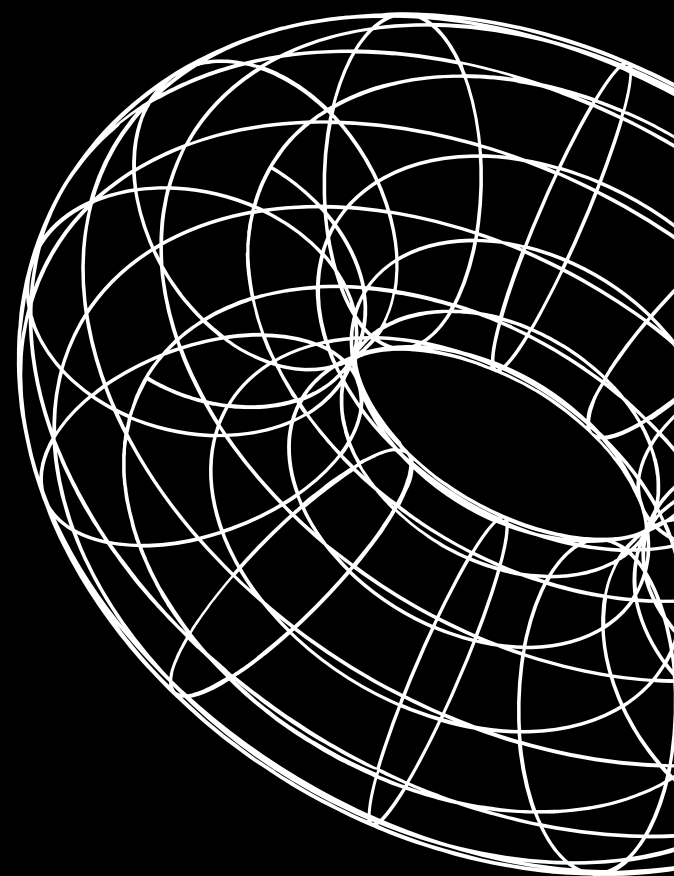
Guardar

Catalogo

ID	Marca	Modelo	Precio	Storage		
1	Xiaomi	redmi	\$500.00	256 gb	Editar	Eliminar



CONCLUSIONES

- Se identificó y clasificó la deuda técnica.
 - Se aplicaron los cuatro tipos de mantenimiento.
 - El sistema incrementó su seguridad y estabilidad.
 - La mantenibilidad y observabilidad mejoraron significativamente.
 - Se redujo el riesgo de errores futuros.
- 



GRACIAS

POR SU ATENCION